

Examen

Jose Eduardo Gonzalez Gutierrez 238494

April 11, 2025

1 Código en C++

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "INFINITUMA09E_2.4";
const char* password = "YxMC9dtMRz";

WebServer server(80);

// Pines del puente H
const int ENA = 7;
const int IN1 = 6;
const int IN2 = 5;
const int ENB = 2;
const int IN3 = 4;
const int IN4 = 3;

// Pines del sensor ultrasónico
const int TRIG = 20;
const int ECHO = 1;

long distancia;

void medirDistancia() {
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    long duracion = pulseIn(ECHO, HIGH);
    distancia = duracion * 0.034 / 2; // Fórmula en cm
}

void detenerMotores() {
    analogWrite(ENA, 0);
}
```

```

    analogWrite(ENB, 0);
}

void setup() {
    Serial.begin(115200);

    // Configuraci n de pines
    pinMode(ENA, OUTPUT);
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(ENB, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);

    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);

    // Conexi n WiFi
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nConexi n WiFi establecida");
    Serial.print("Direcci n IP:");
    Serial.println(WiFi.localIP());

    // Configurar rutas del servidor
    server.on("/adelante", []() {
        medirDistancia();
        if (distancia < 30) {
            detenerMotores();
            server.send(200, "text/plain", "Obst culo detectado:
            motores detenidos");
        } else {
            digitalWrite(IN1, LOW);
            digitalWrite(IN2, HIGH);
            digitalWrite(IN3, LOW);
            digitalWrite(IN4, HIGH);
            analogWrite(ENA, 200);
            analogWrite(ENB, 200);
            server.send(200, "text/plain", "Motores avanzando");
        }
    });

    server.on("/atras", []() {
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        digitalWrite(IN3, HIGH);
        digitalWrite(IN4, LOW);
    });
}

```

```

        analogWrite(ENA, 200);
        analogWrite(ENB, 200);
        server.send(200, "text/plain", "Motores_␣retrocediendo");
    });

    server.on("/girar_derecha", []() {
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        digitalWrite(IN3, LOW);
        digitalWrite(IN4, HIGH);
        analogWrite(ENA, 200);
        analogWrite(ENB, 200);
        server.send(200, "text/plain", "Giro_␣a_␣la_␣derecha");
    });

    server.on("/girar_izquierda", []() {
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, HIGH);
        digitalWrite(IN3, HIGH);
        digitalWrite(IN4, LOW);
        analogWrite(ENA, 200);
        analogWrite(ENB, 200);
        server.send(200, "text/plain", "Giro_␣a_␣la_␣izquierda");
    });

    server.on("/detener", []() {
        detenerMotores();
        server.send(200, "text/plain", "Motores_␣detenidos");
    });

    server.begin();
    Serial.println("Servidor_␣iniciado");
}

void loop() {
    server.handleClient();
}

```

2 Interfaz de movimiento con teclas (WASD) y joystick

```

# -*- coding: utf-8 -*-
"""
Created on Thu Mar 13 14:28:49 2025
@author: usuario
"""

```

```

import tkinter as tk
from PIL import Image, ImageTk
import requests
from pyfirmata import Arduino, util
import threading
import time

# Direcci n IP de la Raspberry Pi Pico W
pico_ip = "192.168.1.110" # Cambia esta IP por la de tu
                          Pico W

# Configuraci n de Arduino
arduino_port = "COM4" # Cambia este puerto seg n tu
                      configuraci n
joystick_activo = False
tecla_presionada = False # NUEVO: bandera para evitar
                          interferencia del joystick

try:
    board = Arduino(arduino_port)
    it = util.Iterator(board)
    it.start()

    # Pines del joystick
    x_pin = board.get_pin('a:3:i') # Eje X
    y_pin = board.get_pin('a:2:i') # Eje Y
    x_pin.enable_reporting()
    y_pin.enable_reporting()
    joystick_activo = True
    print("Joystick conectedo correctamente")
except Exception as e:
    print(f"Error al conectar con Arduino: {e}")
    joystick_activo = False

ultimo_comando = None
zona_muerta = 0.3
umbral_mov = 0.4 # Ajuste del umbral para mayor precisi n
                  en el movimiento

def enviar_comando(comando):
    global ultimo_comando
    if comando != ultimo_comando:
        try:
            url = f"http://{pico_ip}/{comando}"
            response = requests.get(url, timeout=1)
            if response.status_code == 200:
                print(f"Comando {comando} enviado con xito")
                actualizar_estado(comando)
            else:

```

```

        print(f"Error al enviar {comando}: {response
            .status_code}")
    except requests.exceptions.RequestException as e:
        print(f"Error de conexi n: {e}")
        estado.config(text="Error de conexi n", fg="red")
    ultimo_comando = comando

def actualizar_estado(comando):
    estados = {
        "adelante": "Adelante (W/ )",
        "atras": "Atr s (S/ )",
        "girar_derecha": "Derecha (D/ )",
        "girar_izquierda": "Izquierda (A/ )",
        "detener": "Detenido (Espacio)"
    }
    estado.config(text=f"Estado: {estados.get(comando,
        comando})", fg="green")

def manejar_tecla(event):
    global tecla_presionada
    tecla = event.keysym.lower()
    tecla_presionada = True # NUEVO: se activa cuando se
        presiona una tecla

    if tecla == 'w' or event.keysym == "Up":
        enviar_comando("adelante")
    elif tecla == 's' or event.keysym == "Down":
        enviar_comando("atras")
    elif tecla == 'd' or event.keysym == "Right":
        enviar_comando("girar_derecha")
    elif tecla == 'a' or event.keysym == "Left":
        enviar_comando("girar_izquierda")
    elif event.keysym == "space":
        enviar_comando("detener")

def soltar_tecla(event):
    global tecla_presionada
    tecla = event.keysym.lower()
    if tecla in ['w', 'a', 's', 'd'] or event.keysym in ["Up",
        "Down", "Left", "Right"]:
        estado.config(text="Estado: Esperando comando...")
        tecla_presionada = False # NUEVO: se desactiva al
            soltar la tecla

def actualizar_joystick():
    try:
        x_val = x_pin.read()
        y_val = y_pin.read()

```

```

if x_val is not None and y_val is not None:
    x = x_val * 2 - 1
    y = y_val * 2 - 1

    if abs(x) < zona_muerta and abs(y) < zona_muerta:
        enviar_comando("detener")
    else:
        if abs(y) > abs(x):
            if y > umbral_mov:
                enviar_comando("adelante")
            elif y < -umbral_mov:
                enviar_comando("atras")
        else:
            if abs(x) > umbral_mov: # Asegurando
                que cualquier cambio en X sea
                detectado
                if x > 0:
                    enviar_comando("girar_derecha")
                elif x < 0:
                    enviar_comando("girar_izquierda")

except Exception as e:
    print(f"Error leyendo joystick: {e}")

def leer_joystick():
    while joystick_activo:
        if not tecla_presionada: # NUEVO: solo lee joystick
            si no hay tecla presionada
            actualizar_joystick()
            time.sleep(0.1)

def cerrar():
    if joystick_activo:
        board.exit()
        root.destroy()

root = tk.Tk()
root.title("Control de Carro - Teclas y Joystick")
root.geometry("600x700")
root.configure(bg='#f0f0f0')
font_style = ('Arial', 12)

# Instrucciones
instrucciones = tk.Label(root, text="Controles:\nW/Flecha
Arriba-Adelante\nS/Flecha Abajo-Atr s\nA/Flecha
Izquierda-Izquierda\nD/Flecha Derecha-Derecha\
nEspacio-Detener", font=font_style, bg='#f0f0f0')
instrucciones.pack(pady=20)

```

```

estado = tk.Label(root, text="Estado: Esperando comando...",
    font=('Arial', 14, 'bold'), bg='#f0f0f0')
estado.pack(pady=10)

# Imagenes
ruta_flecha_arriba = r"C:\Users\usuario\Documents\Mis programas\Imagenes\up.png"
ruta_flecha_abajo = r"C:\Users\usuario\Documents\Mis programas\Imagenes\down.png"
ruta_flecha_derecha = r"C:\Users\usuario\Documents\Mis programas\Imagenes\right.png"
ruta_flecha_izquierda = r"C:\Users\usuario\Documents\Mis programas\Imagenes\left.png"
ruta_detener = r"C:\Users\usuario\Documents\Mis programas\Imagenes\Stop.png"

try:
    flecha_arriba = ImageTk.PhotoImage(Image.open(
        ruta_flecha_arriba).resize((100, 100)))
    flecha_abajo = ImageTk.PhotoImage(Image.open(
        ruta_flecha_abajo).resize((100, 100)))
    flecha_derecha = ImageTk.PhotoImage(Image.open(
        ruta_flecha_derecha).resize((100, 100)))
    flecha_izquierda = ImageTk.PhotoImage(Image.open(
        ruta_flecha_izquierda).resize((100, 100)))
    imagen_detener = ImageTk.PhotoImage(Image.open(
        ruta_detener).resize((400, 100)))

    marco_controles = tk.Frame(root, bg='#f0f0f0')
    marco_controles.pack(pady=20)

    tk.Button(marco_controles, image=flecha_arriba, command=
        lambda: enviar_comando("adelante")).grid(row=0,
        column=1, padx=5, pady=5)
    tk.Label(marco_controles, text="W/ ", font=font_style,
        bg='#f0f0f0').grid(row=1, column=1)

    tk.Button(marco_controles, image=flecha_izquierda,
        command=lambda: enviar_comando("girar_izquierda")).
        grid(row=1, column=0, padx=5, pady=5)
    tk.Label(marco_controles, text="A/ ", font=font_style,
        bg='#f0f0f0').grid(row=2, column=0)

    tk.Button(marco_controles, image=flecha_derecha, command=
        lambda: enviar_comando("girar_derecha")).grid(row=1,
        column=2, padx=5, pady=5)
    tk.Label(marco_controles, text="D/ ", font=font_style,
        bg='#f0f0f0').grid(row=2, column=2)

```

```

tk.Button(marco_controles, image=flecha_abajo, command=
    lambda: enviar_comando("atras")).grid(row=2, column
        =1, padx=5, pady=5)
tk.Label(marco_controles, text="S/   ", font=font_style,
    bg='#f0f0f0').grid(row=3, column=1)

tk.Button(root, image=imagen_detener, command=lambda:
    enviar_comando("detener")).pack(pady=20)
tk.Label(root, text="ESPACIO", font=font_style, bg='#
    f0f0f0').pack()

except FileNotFoundError as e:
    print(f"Error al cargar im genes:{e}")
tk.Button(root, text="W/   _ _ Adelante", command=lambda:
    enviar_comando("adelante")).pack(pady=5)
tk.Button(root, text="S/   _ _ Atr s", command=lambda:
    enviar_comando("atras")).pack(pady=5)
tk.Button(root, text="A/   _ _ Izquierda", command=lambda
    : enviar_comando("girar_izquierda")).pack(pady=5)
tk.Button(root, text="D/   _ _ Derecha", command=lambda:
    enviar_comando("girar_derecha")).pack(pady=5)
tk.Button(root, text="ESPACIO _ _ Detener", command=lambda
    : enviar_comando("detener")).pack(pady=20)

# Eventos de teclado
root.bind("<KeyPress>", manejar_tecla)
root.bind("<KeyRelease>", soltar_tecla)

# Hilo del joystick
if joystick_activo:
    joystick_thread = threading.Thread(target=leer_joystick,
        daemon=True)
    joystick_thread.start()

root.protocol("WM_DELETE_WINDOW", cerrar)
root.focus_set()
root.mainloop()

```

3 Interfaz

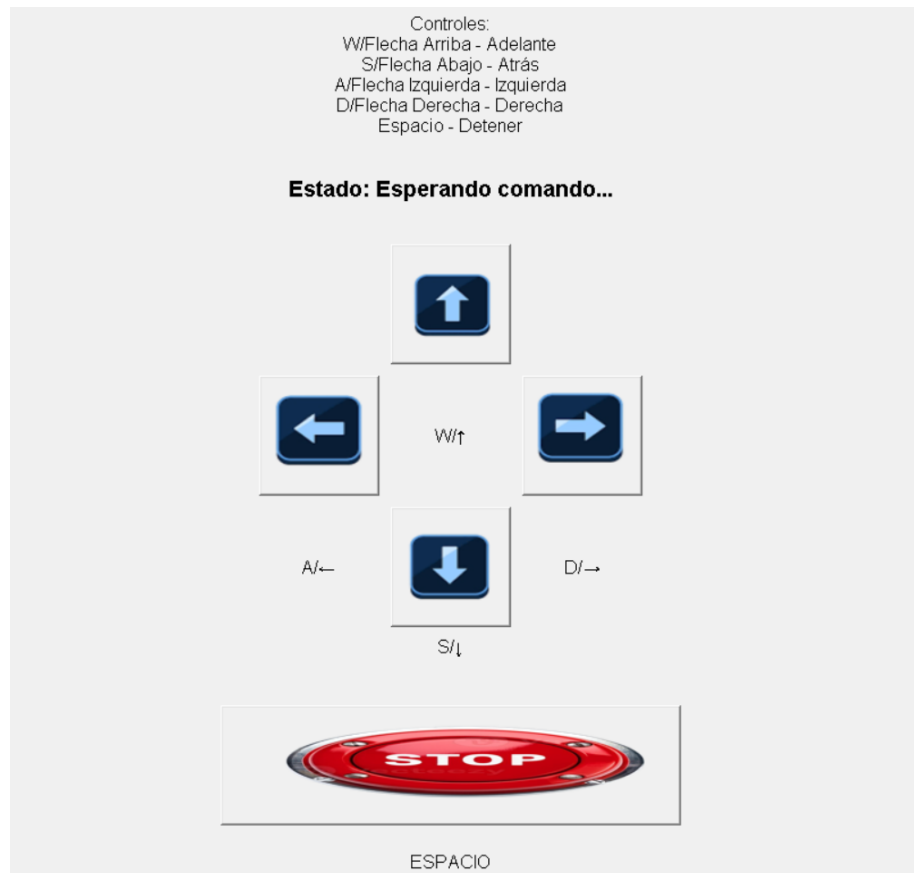


Figure 1: Interfaz

4 Diagrama de conexiones

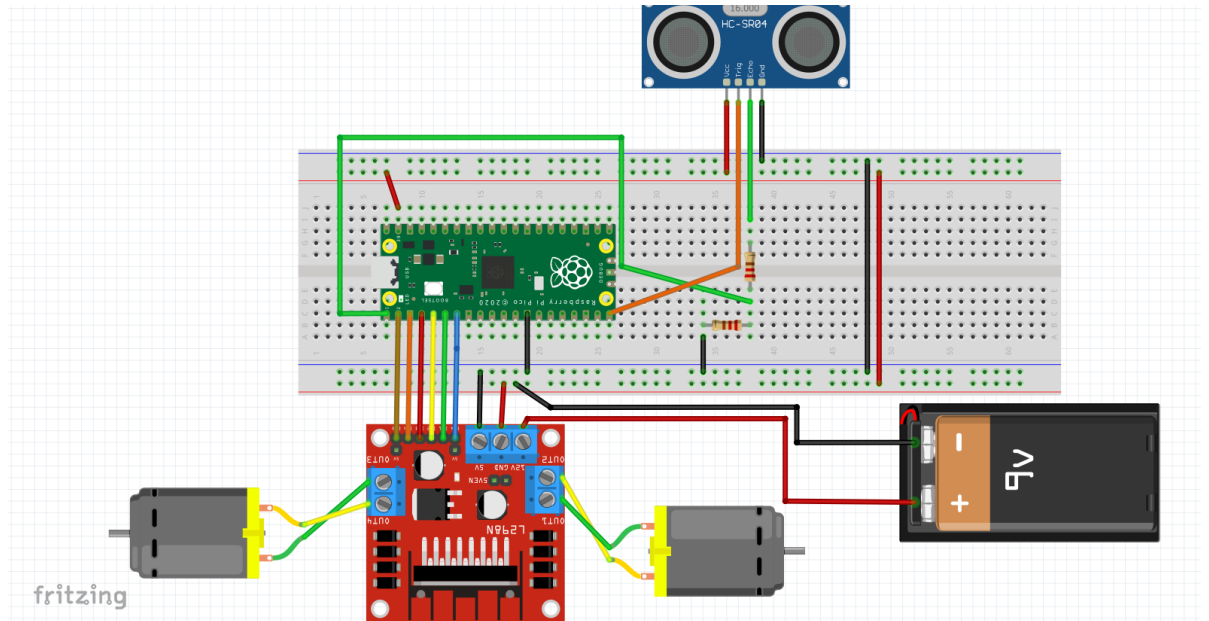
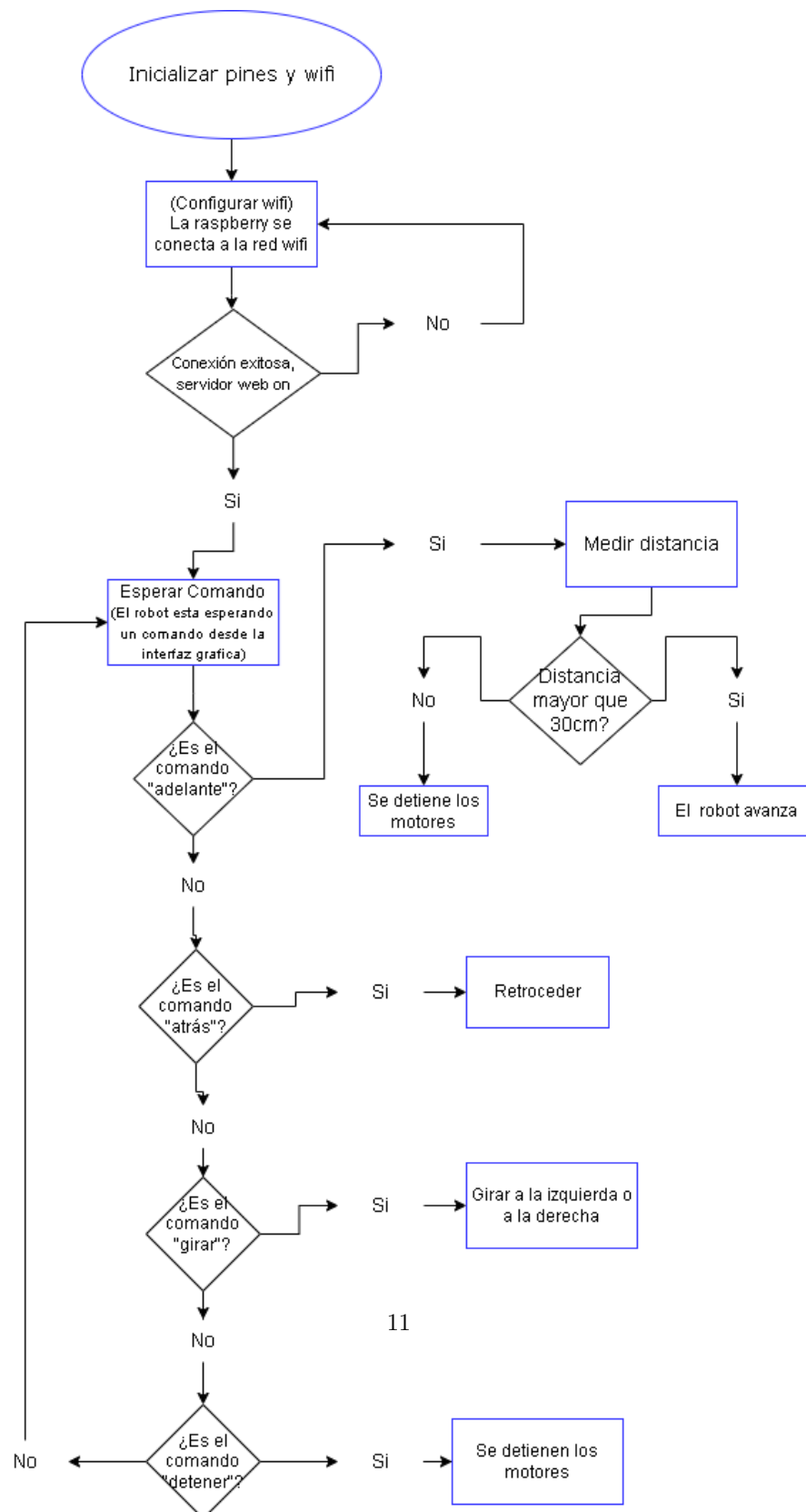


Figure 2: Diagrama de conexiones del circuito

5 Diagrama de flujo



6 Explicación

Para este proyecto, se implementó un control combinado utilizando las teclas WASD, los botones de la interfaz gráfica y un joystick para controlar el movimiento de los motores. Tanto las teclas como el joystick envían comandos desde la interfaz para mover el carro hacia adelante, hacia atrás o para girar, a través de la dirección IP.

En el código se agregó una variable de bloqueo para evitar que las lecturas del joystick interfieran con las teclas. La variable ‘tecla presionada’, en la interfaz gráfica, permite iniciar la lectura del joystick solo cuando ninguna tecla está siendo presionada.

En el caso del control por teclado, el carro se mueve mientras la tecla esté presionada, y los motores se detienen automáticamente al soltarla.

Además, el sensor ultrasónico realiza mediciones únicamente cuando el carro avanza hacia adelante. Si la distancia detectada es igual o menor a 30 cm, los motores se detienen automáticamente para evitar colisiones, pero el carro aún puede girar o retroceder en estas condiciones.